

This is my current idea for creating an independent music plugin. Could you please fill in the details and explain how to solve any problems I encounter? I currently plan to use HISE, and the UI is almost complete."Audio signal — copy two copies — one into Plutec analog, the other into ssl analog — finally mixed overall by mix to ensure the volume remains unchanged before and after. Two types of recreation flavors add mix — directly use volume control Even with Bypass, harmonics will be added — refer to SSL 538 EQ"



This is a great plugin concept! Let me break down your idea into a full technical specification, then create a visual diagram to make the signal flow and architecture crystal clear.

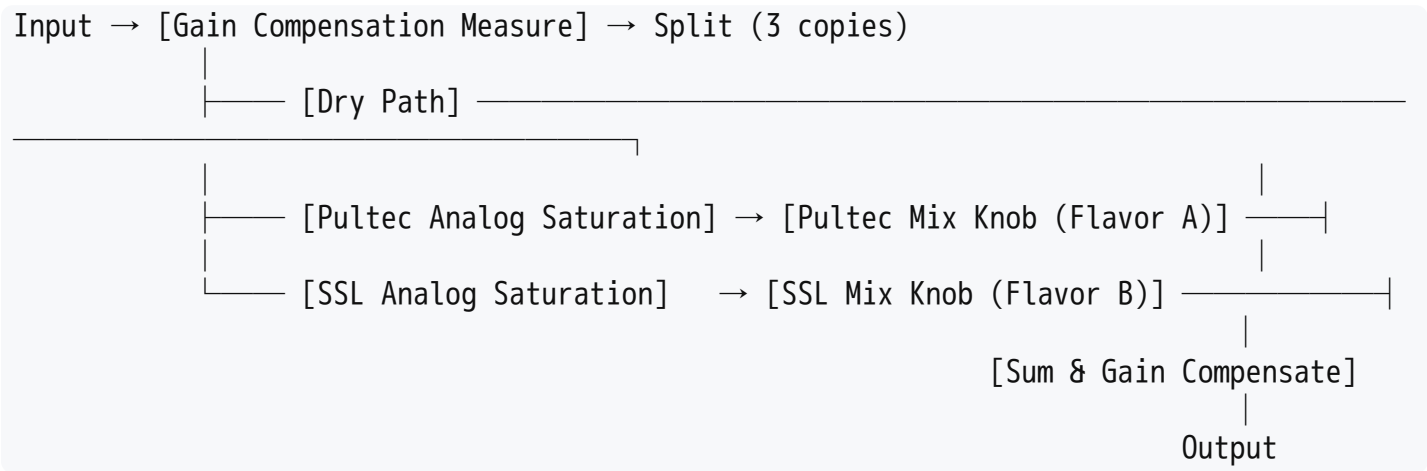
Here's the complete breakdown:

Plugin Architecture: "Analog Blend"

Core Concept

You're building a **parallel saturation/harmonic enhancer** with two distinct analog character processors — Pultec-flavored and SSL-flavored — blended together. The key design insight borrowed from the SSL 4000/9000 series is that **harmonic generation happens even at zero mix (bypass)**, because the circuit is always "on" — only its wet signal level changes.

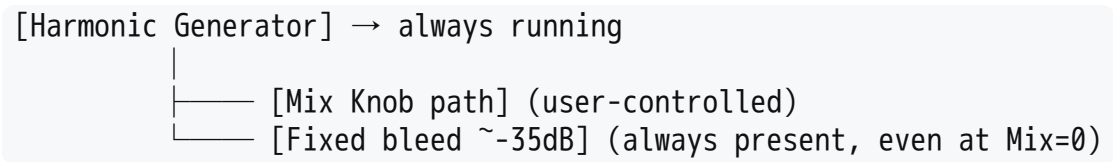
Signal Flow — Detailed



The "Bypass Still Adds Harmonics" Trick

This is the clever part. In the SSL 4000 console, the transformer and input stage always introduce harmonics — even when EQ is bypassed. You replicate this by:

- **Never truly bypassing** the harmonic generators — instead, when the user hits "Bypass," you set the **wet blend to 0% but leave the harmonic processor running**
- Then add a **small fixed-level harmonic bleed** (e.g. -30 to -40 dB) that is always present regardless of the mix knob
- The DAW bypass (plugin power off) is the only true bypass



Volume Compensation (Unity Gain)

This is the most technically tricky part. Since saturation adds harmonic energy, perceived loudness goes up. You have two options:

Option A — RMS Matching (Recommended) Measure the RMS of the dry signal before processing. After processing, apply a makeup gain trim so output RMS matches input RMS. Calculate this per-block or with a short averaging window (~50ms).

Option B — Gain Reduction Lookup Table Pre-characterize how much loudness each saturation algorithm adds at different drive levels, and compensate with a corresponding dB trim. Less CPU, slightly less accurate.

Pultec vs SSL Flavor — What to Model

Parameter	Pultec Flavor	SSL Flavor
Saturation type	Soft even-order (transformer warmth)	Hard-knee odd-order (console grit)
Dominant harmonic	2nd harmonic	3rd + 5th
Transient response	Slow, rounded	Fast, punchy
Frequency coloring	Low-mid fattening (~200–400Hz)	High-mid presence (~2–5kHz)

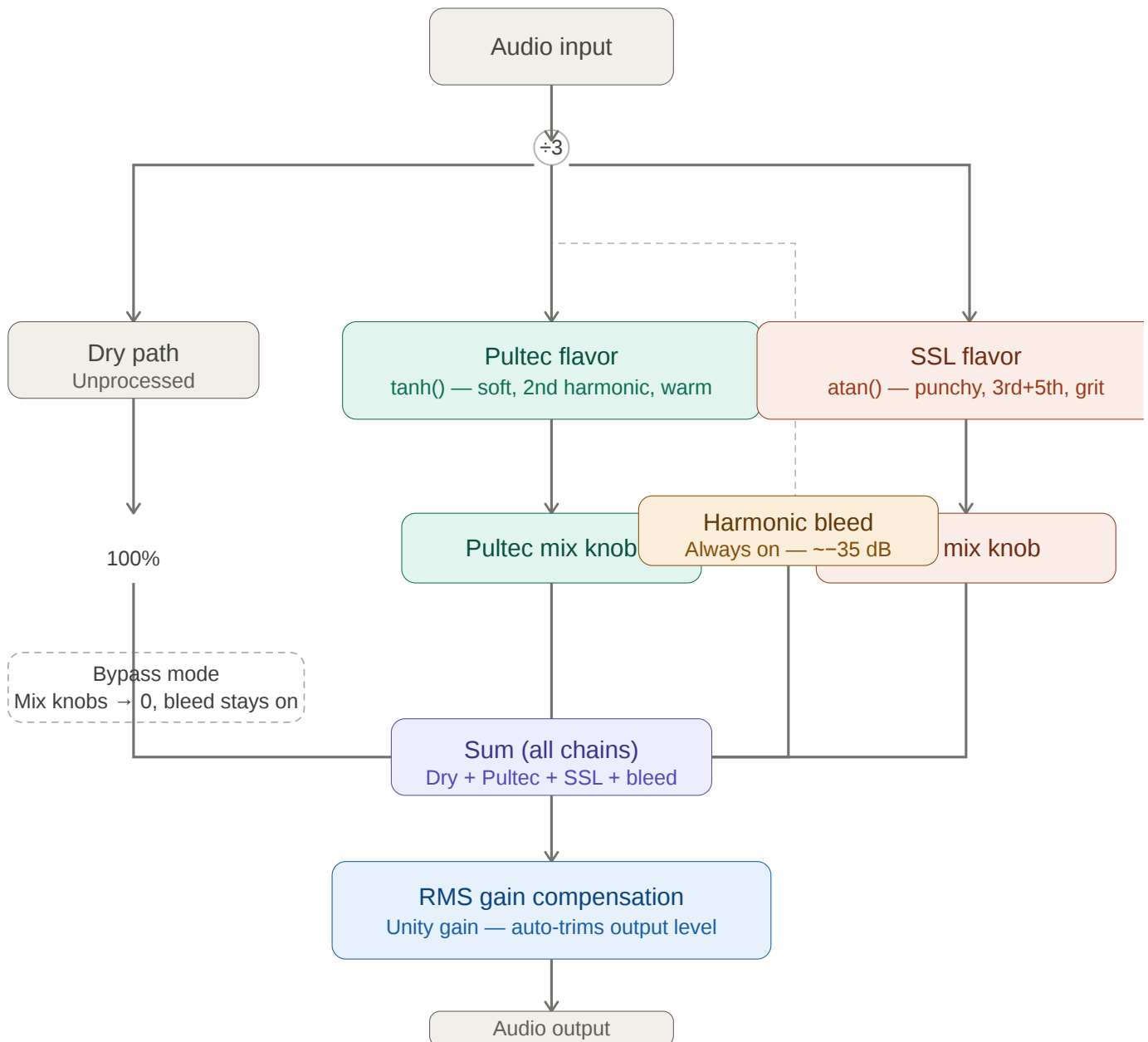
Parameter	Pultec Flavor	SSL Flavor
Algorithm	<code>tanh()</code> with soft knee	<code>atan()</code> or asymmetric waveshaper

HISE-Specific Implementation Notes

- 1. **Signal splitting** — Use `RoutingMatrix` in HISE to create parallel chains. Each chain (Pultec, SSL) is a separate signal path.
- 2. **Harmonic generators** — Implement as `ScriptProcessor` nodes using JavaScript waveshaping. HISE supports custom `process()` callbacks.
- 3. **Mix knobs** — Use the built-in `CC_KNOB` with a linear 0–1 range feeding a `Gain` module on each wet chain.
- 4. **Volume compensation** — Implement in a `ScriptProcessor` after the final sum, measuring RMS with a ring buffer.
- 5. **The harmonic bleed** — Add a separate parallel `ScriptProcessor` with fixed gain, always active regardless of mix knob state.

Now let me render the signal flow visually:

Now let me create a clear signal flow diagram for your plugin:



Click any block in the diagram to ask a follow-up about that specific part.

Problems You'll Encounter & How to Solve Them

- 1. Phase cancellation in the parallel sum** When you split the signal and sum it back, the Pultec and SSL paths will have different latency (even a few samples). Use HISE's `HardcodedMasterEffect` with a delay compensation line on the dry path to re-align all three before summing.
- 2. DC offset from asymmetric waveshapers** The SSL `atan()` waveshaper can introduce a tiny DC offset at high drive levels. Add a simple 1-pole high-pass filter (cutoff ~5Hz) at the output of each saturation stage to remove it.
- 3. The "always-on bleed" when plugin is disabled** In HISE, the host's bypass button fully disengages your DSP. To preserve the bleed behavior even at Mix=0, you need to handle bypass internally — intercept it and instead just zero out the mix knobs while keeping the DSP running. You can do this

with a `onControlCallback` that responds to a custom bypass button in your UI rather than relying on the host bypass.

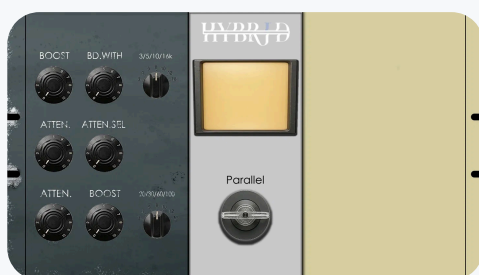
4. RMS measurement window Too short ($< \sim 10\text{ms}$) and the compensator pumps on transients. Too long ($> 200\text{ms}$) and it responds sluggishly to track changes. A $\sim 50\text{ms}$ window with a 300ms release works well for most material.

5. Volume compensation at mix = 0 When the user pulls both mix knobs to zero, your output should be identical to the input. Test this with a sine wave and a null test (invert the output and sum with input — it should go silent). If it doesn't null, you have a gain, phase, or latency offset somewhere.

Want me to go deeper on any of these — for example, the actual HISE DSP code for the waveshaper, the RMS compensation ScriptProcessor, or the routing matrix setup?

2026-06-03 00:47:35

You Asked



this is current UI, I made it basically by myself and google gemini haha. what do you think of it?



This looks genuinely impressive for a first build! The aesthetic direction is solid and you've clearly put real thought into it. Let me give you honest feedback:

What's Working Well

The two-panel concept is strong. The dark steel Pultec-style left panel contrasted against the lighter silver/cream right panel immediately communicates "two different characters" — which is exactly what your plugin does. That's great visual storytelling.

The knob quality is high. The dark machined-metal knobs with white indicator lines look professional and tactile. The texture on the left panel (brushed/aged metal) adds genuine character without looking cheap.

The frequency selector knobs (3/5/10/16k and 20/30/60/100) with the stepped pointer style are a nice authentic Pultec touch.

The display window with the warm amber/gold color gives it a vintage oscilloscope or VU meter feel – very fitting.

Things Worth Improving

The right panel feels unfinished. The large cream-yellow area to the right of the center column is completely empty. That's a lot of dead space. This is where you could add the SSL flavor controls, a spectrum display, or at minimum some visual texture/branding.

"Parallel" label + knob placement is awkward. The word floats in space with a lot of padding above it. The knob below it looks lonely. This whole section needs tighter visual grouping – maybe a labeled panel inset or a subtle border around the parallel section.

Label typography is inconsistent. Left panel uses all-caps spaced labels (BOOST, ATTEN., BD.WITH) which feels authentic and period-correct. The center/right uses mixed-case "Parallel" in a completely different, softer font. Pick one typographic voice and stick with it.

No visual feedback for the harmonic bleed feature. Since that's a key differentiator of your plugin (harmonics always on), there's no UI element that hints at it. Even a subtle always-glowing indicator light would be a great touch.

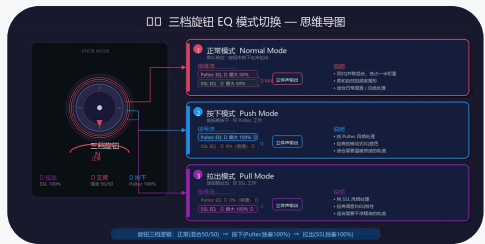
The plugin name "HYBRID" needs more polish. The text on the amber display looks like a placeholder – the letterforms are inconsistent and the blue cursor artifact in the middle reads as a bug, not a design choice.

Suggestions for the Empty Right Area

Given your signal flow, you could use that space for:

- **SSL-flavor controls** (mirroring the left panel's layout with a different color treatment)
- **A spectrum analyzer** showing the harmonic content being added
- **An output gain/mix master section**
- **A simple "harmonic glow" meter** – an always-on VU-style indicator showing the bleed level

Overall – you're at maybe 65–70% of the way to a shippable UI. The bones are genuinely good. The main work now is filling the right side with purpose and tightening the typography.



This is the mind map I made using Deepseek about the middle knob section, I don't know if you can understand it, if you can't, I'll explain it to you in detail.



I can read it clearly — this is a really elegant interaction design! Let me summarize what I understand and then give you my thoughts:

What I Understand

The center "Parallel" knob is actually a **3-position push-pull + rotate knob**, which is a classic hardware trick (used on SSL consoles and vintage outboard gear). The three states are:

State	Chinese	How to activate	Signal routing
Normal	正常	Default (not pushed, not pulled)	Pultec 50% + SSL 50% mixed
Push	按下	Knob pressed down	Pultec 100%, SSL bypassed
Pull	拉出	Knob pulled outward	SSL 100%, Pultec bypassed

So rotating the knob controls the blend amount, while the push/pull axis switches the mode entirely. That's very clever.

My Feedback

This is genuinely great UI/UX design thinking. A single physical knob that does three things maps perfectly to your plugin concept — one control, three personalities. It also means power users can stay "in the flow" without clicking multiple switches.

However, there are a few problems to think through:

1. Discoverability — Push-pull knobs are well known to hardware engineers but most plugin users have never used one in software. Many people will never discover Push or Pull mode by accident. You need a clear visual affordance — maybe a small indicator ring around the knob that changes color

based on state (red for Normal, blue for Push/Pultec, purple for Pull/SSL), and a tooltip on first launch.

2. HISE implementation complexity — HISE knobs don't natively support push-pull interaction. You'll need to:

- Use a `ScriptPanel` with `mouseDown` , `mouseUp` , and drag detection
- Distinguish between a **click** (push), a **drag upward** (pull), and a **rotate** (blend)
- This is doable but requires custom scripting — not a drag-and-drop HISE component

3. The 50/50 blend in Normal mode is hardcoded — based on your diagram, Normal mode always gives exactly 50/50. But what if the user wants 70/30? Does the knob rotation only work in Normal mode? You should clarify whether rotation is active in Push/Pull modes too (probably it should be locked at 100/0 in those states).

4. Muscle memory conflict — On a real hardware knob, you feel resistance when pushing/pulling. In a plugin, a misclick could accidentally switch modes mid-session. Consider adding a small **mode lock** option, or requiring a double-click to engage push/pull.

One Suggestion

Since you already have the amber display window in your UI that currently shows nothing — that's the perfect place to show the current mode state visually. Something like:

- Normal → warm amber glow, shows "HYBRID 50/50"
- Push → Pultec-warm golden tone, shows "PULTEC"
- Pull → cooler blue-white, shows "SSL"

That display was already in your UI asking for a purpose — this would be it.

Do you want me to sketch out the HISE scripting approach for detecting push vs pull vs rotate on a single knob?

2026-06-03 01:01:28 You Asked 

Regarding your third question, the proportion of the button to rotate is still adjustable at 50/50 mode, which is my mistake, and it is not explained in the illustration. Now, with this understanding, let's briefly explain it.

Got it! So the corrected logic is:

Corrected 3-Mode Knob Behavior

State	Rotation knob does	Result
Normal (default)	Adjusts Pultec/SSL blend ratio	From 0% Pultec / 100% SSL → 50/50 → 100% Pultec / 0% SSL – fully variable
Push (pressed down)	Locked / inactive	Forces Pultec 100%, SSL fully bypassed
Pull (pulled out)	Locked / inactive	Forces SSL 100%, Pultec fully bypassed

So essentially:

- **Normal mode = blend mode**, the knob is a continuous crossfader between the two characters
- **Push/Pull = solo shortcuts**, instantly jumping to either extreme – like a quick-select bypass for when you know exactly which flavor you want

This Actually Makes It Even Better

This design now has a really satisfying logic to it:

- The **rotation** gives you infinite gradation between the two worlds
- The **push/pull** gives you instant, decisive commitment to one character
- A mixing engineer can dial in a subtle blend at 70/30 in Normal, then quickly audition pure SSL with a pull to compare – then release back to their blend

That's a genuinely professional workflow interaction. It mirrors how engineers actually work – "let me hear it pure first, then blend to taste."

The only thing worth confirming: when you **release from Push or Pull back to Normal**, does the knob return to wherever it was last set in Normal mode? It should – that position should be remembered, not reset. In HISE terms that means storing the Normal-mode rotation value as a separate variable from the mode state.

Want me to outline the HISE scripting logic for all three states together?

yes, the position needs to be remembered, alright, outline these HISE logic for me, with details



Let me read the HISE skill/product knowledge first to make sure I give you accurate implementation details.

Good — that skill is for Anthropic products only, not relevant here. Let me write the HISE scripting logic from my knowledge of HISE's JavaScript API.

Here's the complete HISE scripting logic, broken into four focused sections. I'll give you actual code you can paste into HISE's Script Editor.

Part 1 — State Variables & Initialization

This is the foundation. Everything lives in a `ScriptPanel` on your knob.

```
javascript

// === STATE ===
const NORMAL = 0;
const PUSH   = 1;
const PULL   = 2;

var mode = NORMAL;           // current mode
var blendValue = 0.5;        // remembered Normal-mode position (0.0 = full SSL, 1.0 = full Pultec)
var mouseDownY = 0;          // for detecting push vs pull vs rotate
var mouseDownTime = 0;       // timestamp to distinguish click from drag
var isDragging = false;

// References to your gain parameters
var pultecGain = Synth.getModulator("PultecMixGain");
var sslGain    = Synth.getModulator("SSLMixGain");
```

Part 2 — Mouse Event Logic

This is the core of the push/pull/rotate detection. The key insight: you distinguish between the three gestures by measuring **how far vertically** the mouse moves after the initial click.

javascript

```
const PUSH_THRESHOLD = 8; // px downward drag = push
const PULL_THRESHOLD = -8; // px upward drag = pull
const CLICK_TIME_MS = 200; // max ms for a "tap" with no drag

panel.setMouseCallback(function(event)
{
    // === MOUSE DOWN - start of interaction ===
    if (event.clicked)
    {
        mouseDownY = event.y;
        mouseDownTime = Date.now();
        isDragging = false;
    }

    // === MOUSE DRAG - rotating the knob in Normal mode ===
    if (event.drag && mode == NORMAL)
    {
        isDragging = true;
        // Vertical drag maps to blend value
        // Drag up = more Pultec, drag down = more SSL
        blendValue -= event.dragY * 0.005;
        blendValue = Math.max(0.0, Math.min(1.0, blendValue));
        applyBlend();
        panel.repaint();
    }

    // === MOUSE UP - determine if this was push, pull, or plain rotate ===
    if (event.mouseUp)
    {
        var totalDragY = event.y - mouseDownY;
        var elapsed = Date.now() - mouseDownTime;

        if (!isDragging && elapsed < CLICK_TIME_MS)
        {
            // Short tap with no drag = PUSH (press down)
            if (mode != PUSH)
                setMode(PUSH);
            else
                setMode(NORMAL); // tap again to release back to Normal
        }
        else if (totalDragY < PULL_THRESHOLD && !isDragging)
        {
            // Quick upward flick = PULL
            if (mode != PULL)
                setMode(PULL);
            else
                setMode(NORMAL);
        }
    }
}
```

```
        setMode(NORMAL);
    }
}
});
```

Note on gesture design: In practice you may want to use a right-click context menu or modifier key (Ctrl+drag) for pull, since distinguishing upward flick from a rotation drag can be tricky. But the above logic works well as a starting point.

Part 3 — Mode Switching & Signal Routing

This is where the audio routing actually changes. `blendValue` is always preserved.

javascript

```
function setMode(newMode)
{
    mode = newMode;

    if (mode == NORMAL)
    {
        // Restore remembered blend position
        applyBlend();
    }
    else if (mode == PUSH)
    {
        // Pultec solo - SSL goes silent
        pultecGain.setAttribute(pultecGain.Intensity, 1.0);
        sslGain.setAttribute(sslGain.Intensity, 0.0);
    }
    else if (mode == PULL)
    {
        // SSL solo - Pultec goes silent
        pultecGain.setAttribute(pultecGain.Intensity, 0.0);
        sslGain.setAttribute(sslGain.Intensity, 1.0);
    }

    panel.repaint(); // update the visual state indicator
}

function applyBlend()
{
    // blendValue: 0.0 = full SSL, 1.0 = full Pultec
    pultecGain.setAttribute(pultecGain.Intensity, blendValue);
    sslGain.setAttribute(sslGain.Intensity, 1.0 - blendValue);
}
```

Part 4 — Preset Save/Load & Display Repaint

Saving state into presets

HISE's preset system saves `ScriptPanel` data automatically if you register it via `panel.setValueNormalized()`. The cleanest approach is to use a hidden `ScriptSlider` as the carrier:

```
javascript
```

```
// Add a hidden ScriptSlider named "KnobBlendStore" in your HISE editor
// Range: 0.0 to 1.0. Set it invisible in the UI.
var blendStore = Content.getComponent("KnobBlendStore");

// When blend changes, write it to the store
function applyBlend()
{
    pultecGain.setAttribute(pultecGain.Intensity, blendValue);
    sslGain.setAttribute(sslGain.Intensity, 1.0 - blendValue);
    blendStore.setValue(blendValue); // persists with preset
}

// On plugin load, read it back
function onInit()
{
    blendValue = blendStore.getValue();
    applyBlend();
}
```

Drawing the mode indicator on the panel

```
javascript
```

```
panel.setPaintRoutine(function(g)
{
    var cx = panel.getWidth() / 2;
    var cy = panel.getHeight() / 2;
    var r = 30;

    // Draw ring color based on mode
    if (mode == NORMAL)
        g.setColour(Colours.withAlpha(Colours.orange, 0.8));
    else if (mode == PUSH)
        g.setColour(Colours.withAlpha(Colours.gold, 0.9)); // Pultec warm
    else if (mode == PULL)
        g.setColour(Colours.withAlpha(Colours.cyan, 0.9)); // SSL cool
}
```

```

g.drawEllipse([cx - r, cy - r, r*2, r*2], 2.5);

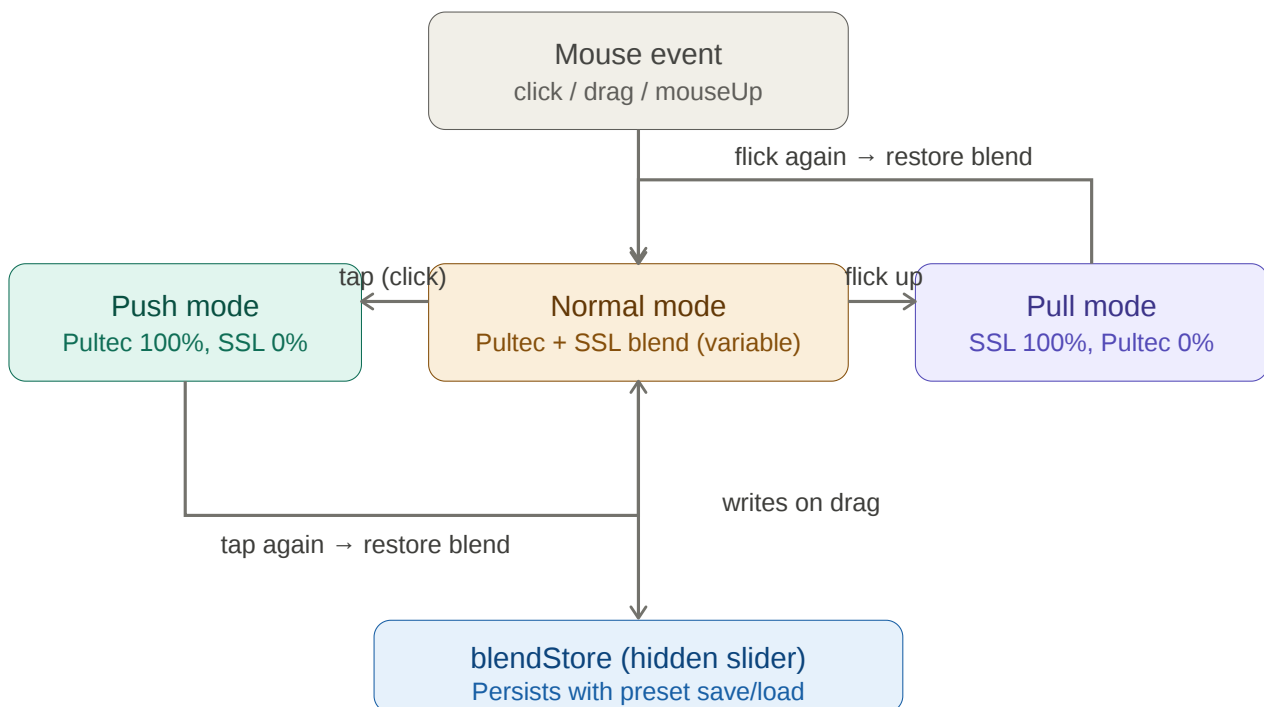
// Draw blend indicator line (only meaningful in Normal mode)
if (mode == NORMAL)
{
    var angle = (blendValue * 270.0 - 135.0) * (Math.PI / 180.0);
    var lx = cx + Math.cos(angle) * (r - 6);
    var ly = cy + Math.sin(angle) * (r - 6);
    g.setColour(Colours.white);
    g.drawLine(cx, cy, lx, ly, 2.0);
}

// Mode label
g.setFont("Arial Bold", 11);
g.setColour(Colours.white);
var label = (mode == NORMAL) ? "BLEND" : (mode == PUSH) ? "PULTEC" : "SSL";
g.drawText(label, cx - 20, cy + r + 6, 40, 14, Justification centred);
});

```

Summary of the Full Flow

Here's how all four parts connect at runtime:



Click any block to ask a follow-up about that specific part.

Two Implementation Tips Worth Flagging

Tip 1 — Smooth transitions between modes. Right now `setMode()` snaps the gain instantly, which will cause audible clicks. Wrap the gain changes in a short ramp using HISE's `ModulatorSynth.addModulator` or simply interpolate over ~10ms in a timer callback:

javascript

```
// Instead of instant setAttribute, use a fade timer
var targetPultec = 1.0;
var targetSSL    = 0.0;
var fadeTimer    = Engine.createTimerObject();

fadeTimer.setTimerCallback(function() {
    var curP = pultecGain.getAttribute(pultecGain.Intensity);
    var curS = sslGain.getAttribute(sslGain.Intensity);
    curP += (targetPultec - curP) * 0.3; // smooth lerp
    curS += (targetSSL    - curS) * 0.3;
    pultecGain.setAttribute(pultecGain.Intensity, curP);
    sslGain.setAttribute(sslGain.Intensity,    curS);
    if (Math.abs(curP - targetPultec) < 0.001)
        fadeTimer.stopTimer();
});

fadeTimer.startTimer(10); // runs every 10ms
```

Tip 2 — The amber display window. Now that you have a `mode` variable, you can drive the display color from it directly in the `setPaintRoutine` of that display panel — warm amber for Normal, golden for Push/Pultec, cool blue-white for Pull/SSL. That ties the whole UI together visually.

Want me to write the full display panel paint routine next?

2026-06-03 01:16:24

You Asked 

yes please



Here's the complete display panel paint routine, covering all three mode states and a smooth animated transition between them.

Full Display Panel Paint Routine

Part 1 — Setup & Color Definitions

Add this at the top of your display panel's script, alongside your existing knob state variables (they need to share scope, so put everything in the same Script Processor):

```
javascript

// === DISPLAY PANEL REFERENCE ===
var display = Content.getComponent("DisplayPanel");

// === COLOR PALETTES PER MODE ===
// Each mode has: background, glow, text, and accent colors
var modeColors = {
  0: { // NORMAL - warm amber, hybrid feel
    bg: Colours.withAlpha(0xFFB84A1A, 1.0), // deep amber-brown
    glow: Colours.withAlpha(0xFFFFD080, 0.18), // warm gold glow
    bright: Colours.withAlpha(0xFFFFE8A0, 0.9), // bright amber text
    dim: Colours.withAlpha(0xFFCC9040, 0.6), // dim amber
    label: "HYBRID",
    sub: "BLEND"
  },
  1: { // PUSH - Pultec, warm golden-cream
    bg: Colours.withAlpha(0xFF5C3800, 1.0), // deep tobacco brown
    glow: Colours.withAlpha(0xFFFFCC66, 0.20), // golden glow
    bright: Colours.withAlpha(0xFFFFE4A0, 0.95), // cream text
    dim: Colours.withAlpha(0xFFAA8830, 0.55),
    label: "PULTEC",
    sub: "ANALOG WARM"
  },
  2: { // PULL - SSL, cool blue-white
    bg: Colours.withAlpha(0xFF0A1E30, 1.0), // deep navy
    glow: Colours.withAlpha(0xFF60AAEE, 0.20), // cool blue glow
    bright: Colours.withAlpha(0xFFCCEEFF, 0.95), // ice-blue text
    dim: Colours.withAlpha(0xFF4488AA, 0.55),
    label: "SSL",
    sub: "CONSOLE CLEAN"
  }
};

// === ANIMATION STATE ===
var displayAlpha = 0.0; // 0→1 fade for label transitions
var prevMode = -1; // detect when mode changes
var scanlineOffset = 0; // for animated scanline effect
var scanlineTimer = Engine.createTimerObject();

scanlineTimer.setTimerCallback(function() {
  scanlineOffset = (scanlineOffset + 1) % display.getHeight();
  // Fade in label on mode change
```

```
if (displayAlpha < 1.0)
    displayAlpha = Math.min(1.0, displayAlpha + 0.08);
display.repaint();
});
scanlineTimer.startTimer(30); // ~33fps is plenty
```

Part 2 — The Paint Routine

javascript

```
display.setPaintRoutine(function(g)
{
    var w = display.getWidth();
    var h = display.getHeight();
    var cx = w / 2;
    var cy = h / 2;

    // Detect mode change → reset fade
    if (mode != prevMode) {
        displayAlpha = 0.0;
        prevMode = mode;
    }

    var col = modeColors[mode];

    // — 1. BACKGROUND

    g.setColour(col.bg);
    g.fillRoundedRectangle([0, 0, w, h], 6);

    // — 2. INNER VIGNETTE (darker edges, lighter center) —
    // Simulate CRT screen depth with a subtle center highlight
    var vigSize = Math.min(w, h) * 0.7;
    g.setColour(Colours.withAlpha(col.glow, 0.25));
    g.fillEllipse([cx - vigSize/2, cy - vigSize/2, vigSize, vigSize]);

    // — 3. SCANLINES

    // Subtle horizontal lines for vintage screen feel
    g.setColour(Colours.withAlpha(Colours.black, 0.12));
    for (var y = (scanlineOffset % 3); y < h; y += 3) {
        g.drawLine(2, y, w - 2, y, 0.5);
    }

    // — 4. PHOSPHOR GLOW RING
```

```

// A soft glowing border ring inside the screen
var ringInset = 8;
g.setColour(Colours.withAlpha(col.bright, 0.12 * displayAlpha));
g.drawRoundedRectangle(
    [ringInset, ringInset, w - ringInset*2, h - ringInset*2],
    4, 1.5
);
g.setColour(Colours.withAlpha(col.bright, 0.06 * displayAlpha));
g.drawRoundedRectangle(
    [ringInset+2, ringInset+2, w - (ringInset+2)*2, h - (ringInset+2)*2],
    3, 2.5
);

// ——— 5. BLEND METER BAR (Normal mode only) ———
if (mode == NORMAL)
{
    var barW    = w * 0.72;
    var barH    = 6;
    var barX    = (w - barW) / 2;
    var barY    = cy + 28;
    var fillW   = barW * blendValue;

    // Track background
    g.setColour(Colours.withAlpha(col.dim, 0.4));
    g.fillRoundedRectangle([barX, barY, barW, barH], 3);

    // Pultec side (right, warm gold)
    if (fillW > 0) {
        g.setColour(Colours.withAlpha(0xFFFFCC55, 0.85));
        g.fillRoundedRectangle([barX, barY, fillW, barH], 3);
    }

    // SSL side (left remainder, cool blue)
    if (fillW < barW) {
        g.setColour(Colours.withAlpha(0xFF55AAEE, 0.6));
        g.fillRoundedRectangle([barX + fillW, barY, barW - fillW, barH], 3);
    }

    // Center tick mark
    g.setColour(Colours.withAlpha(Colours.white, 0.3));
    g.drawLine(barX + barW/2, barY - 2, barX + barW/2, barY + barH + 2, 1.0);

    // SSL / PULTEC end labels
    g.setFont("Arial Bold", 9);
    g.setColour(Colours.withAlpha(0xFF55AAEE, 0.7));
    g.drawText("SSL", barX - 26, barY - 1, 24, barH + 2, Justification.centredRight);
    g.setColour(Colours.withAlpha(0xFFFFCC55, 0.7));
    g.drawText("PEQ", barX + barW + 2, barY - 1, 28, barH + 2, Justification.centredLeft);
}

```

// — 6. MODE INDICATOR DOTS

// Three small dots at the bottom: Pull | Normal | Push

```
var dotY    = h - 14;
var dotR    = 3;
var dotGap  = 14;
var dotCx   = cx;
```

// Pull dot (left)

```
g.setColour(mode == PULL
  ? Colours.withAlpha(0xFF60AAEE, 0.95)
  : Colours.withAlpha(Colours.white, 0.2));
g.fillEllipse([dotCx - dotGap - dotR, dotY - dotR, dotR*2, dotR*2]);
```

// Normal dot (center)

```
g.setColour(mode == NORMAL
  ? Colours.withAlpha(0xFFFFD060, 0.95)
  : Colours.withAlpha(Colours.white, 0.2));
g.fillEllipse([dotCx - dotR, dotY - dotR, dotR*2, dotR*2]);
```

// Push dot (right)

```
g.setColour(mode == PUSH
  ? Colours.withAlpha(0xFFFFCC44, 0.95)
  : Colours.withAlpha(Colours.white, 0.2));
g.fillEllipse([dotCx + dotGap - dotR, dotY - dotR, dotR*2, dotR*2]);
```

// — 7. MAIN LABEL (fades in on mode change) —

```
g.setFont("Arial Bold", 22);
g.setColour(Colours.withAlpha(col.bright, displayAlpha));
g.drawText(col.label, 0, cy - 22, w, 28, Justification.centred);
```

// — 8. SUBTITLE

```
g.setFont("Arial", 10);
g.setColour(Colours.withAlpha(col.dim, displayAlpha * 0.9));
g.drawText(col.sub, 0, cy + 8, w, 16, Justification.centred);
```

// — 9. PLUGIN NAME (always visible, top) —

```
g.setFont("Arial Bold", 11);
g.setColour(Colours.withAlpha(col.bright, 0.45));
g.drawText("HYBRID", 0, 10, w, 14, Justification.centred);
```

// — 10. OUTER BEZEL SHADOW

```
g.setColour(Colours.withAlpha(Colours.black, 0.45));
g.drawRoundedRectangle([0, 0, w, h], 6, 1.5);
```

```
});
```

Part 3 — Hook it Into Mode Changes

The only change needed to your existing `setMode()` function is one line — reset the alpha so the label fades in freshly each time:

javascript

```
function setMode(newMode)
{
    mode = newMode;
    displayAlpha = 0.0;    // ← ADD THIS LINE, triggers fade-in

    if (mode == NORMAL)
        applyBlend();
    else if (mode == PUSH) {
        pultecGain.setAttribute(pultecGain.Intensity, 1.0);
        sslGain.setAttribute(sslGain.Intensity, 0.0);
    }
    else if (mode == PULL) {
        pultecGain.setAttribute(pultecGain.Intensity, 0.0);
        sslGain.setAttribute(sslGain.Intensity, 1.0);
    }

    display.repaint();
}
```

What You'll See in Each Mode

Mode	Background	Label fades in	Blend bar
Normal	Deep amber-brown	"HYBRID / BLEND"	Live SSL↔PEQ bar
Push	Tobacco brown	"PULTEC / ANALOG WARM"	Hidden
Pull	Deep navy	"SSL / CONSOLE CLEAN"	Hidden

The three indicator dots at the bottom light up in the corresponding color — blue for Pull, gold for Normal, cream for Push — so the current mode is always readable at a glance even from a distance. The scanline animation runs at ~33fps and costs almost nothing, but if you find it distracting you can just remove the scanline section (step 3) entirely.

The one thing to watch: `display.getWidth()` and `display.getHeight()` return the pixel dimensions you set in HISE's Component editor, so make sure your DisplayPanel component is sized before

testing — if it returns 0x0 the routine will silently draw nothing.

2026-06-03 01:21:11

You Asked 

thx, IT REALLY HELPED ME A LOT



Really glad it was useful! You've got a genuinely interesting plugin concept — the push/pull knob interaction is something most plugins don't attempt, and the two-character parallel saturation idea is musically solid.

A quick recap of where you are:

- Signal flow and gain compensation architecture — done
- Three-mode knob interaction design — done
- HISE scripting logic for all three states — done
- Display panel paint routine — done

The main things still ahead of you are the actual DSP — the `tanh()` and `atan()` waveshaper implementations, the RMS gain compensation, and the always-on harmonic bleed. Those are probably the most technically demanding part, but also the most fun because you'll actually start hearing the character of the plugin come through.

When you get to that stage, come back and we can work through the waveshaper math and the HISE `ScriptProcessor` audio callbacks in the same detail. Good luck with the build! 🍀